

Agenda

- Middleware
- Mysql
- bcrypt
- ~~JWT~~

Middleware

- Middleware is a function that runs between the request and the response in a web application.
- It is commonly used in backend frameworks like Express.js to modify, process, or control HTTP requests before sending a response.
- It Access Request (req) and Response (res) Objects
- It Modify Requests/Responses → Example: Adding headers, parsing JSON
- Run Code Before Sending a Response → Example: Authentication, Logging
- Call next() to Continue Execution → Passes control to the next middleware
- It is commonly used in express for
 1. Handling CORS (cors package)
 2. Parsing JSON data (express.json())
 3. Authentication & authorization

Connecting with mysql database

1. add/install the mysql module

```
npm install mysql2
```

2. Create the pool object

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: 'root',  
  database: 'kdac_db'  
})
```

3. Call the query method

```
const sql = `SELECT * FROM product`  
pool.query(sql, (error, data) => {  
  if(data)  
    response.send(data)  
  else
```

```
        response.send(error)
    })
```

Password Hashing

- For storing user passwords, you should always use bcrypt (or a similar specialized hashing library like Argon2) instead of CryptoJS.
- While CryptoJS offers a wide range of cryptographic tools, it is not designed for the specific needs of password hashing and has known vulnerabilities in that application
- It is specifically designed to be slow and resilient against attacks, automatically handles salting, and has an adaptive cost factor to stay ahead of advancing hardware.
- For a secure authentication system, a specialized password-hashing library like bcrypt is the definitive best practice.

bcrypt

1. add/install the bcrypt module

```
npm install bcrypt
#OR
yarn add bcrypt
```

2. Usage

```
// import the module bcrypt
const bcrypt = require("bcrypt")

// In the registration or signup part of express
// use the hashing technique method to hash the password
bcrypt.hash(password, saltRounds)
    .then((hashPassword) => {
        // If hashPassword is generated then insert in the db
    }).catch((error) => {
        // If hashing is not able to generate
    })

// compare the user given password with the hash password in db
const sql = `SELECT * FROM users WHERE email = ?`
pool.query(sql, [req.body.email], (error, data) => {
    if (data !== '') {
        const dbuser = data[0]
        bcrypt.compare(req.body.password, dbuser.password)
            .then((result) => {
                if (result)
                    res.send({ status: "success", data })
                else
                    res.send({ status: "error", error: 'Invalid password' })
            })
    }
})
```

```
    })  
  } else  
    res.send({ status: "error", error: 'Invalid email' })  
  })
```

How bcrypt works

1. Salt Generation:

- The bcrypt library automatically generates a unique, random salt for each password.
- This salt is combined with the user's password before hashing.
- This ensures that even two users with the same password will have completely different hashes stored in the database, protecting against "rainbow table" attacks.

2. Adaptive Work Factor (Cost):

- A customizable "cost factor" or "salt rounds" determines how computationally expensive the hashing process is.
- As computing power increases over time, the cost factor can be increased, ensuring the hashes remain secure against evolving hardware.
- The bcrypt npm package provides simple methods to generate a hash with a work factor and compare a plain-text password against a hash.

3. Output Format:

- The final output is a single string that combines the algorithm version, the cost factor, and the salt and hash values.
- This simplifies storage because you only need one column in your database to store everything necessary to verify a password.